Scientific Protocol: SPI-Based LED Bar Graph Control Using EFM8 Microcontroller

Contents

1 Abstract **3**

2 Introduction **4**

2.1 The Serial Peripheral Interface (SPI) 4

2.2 The Shift Register 7

2.3 Bar Graph Click – Hardware and Shift Register Operation . . 7

2.4 Project Objectives 9

3 Materials and Methods **10**

3.1 Hardware Components 10

3.2 Software Components 10

3.3 Methodology 10

3.4 Hardware Configuration & Pin Mapping 11

4 Result **13**

4.1 Pattern 1: Heartbeat Pattern 13

4.2 Pattern 2: Bouncing Light 14

4.3 Firmware Operation of `barcontrol.c` 15

5 Discussion **18**

1 Abstract

This project presents the design and implementation of a firmware-based SPI communication system between the Silicon Labs EFM8BB52 microcontroller and a 10-LED Bar Graph Click module. The goal is to demonstrate the proper setup and functionality of the SPI protocol and GPIO-based peripheral control. The system sends serial data through SPI to control the LED bar display. The project validates correct communication by displaying dynamic light patterns, confirming the integration of software modules and hardware interfaces.

2 Introduction

Microcontrollers are the core of modern digital control systems, but their usefulness depends on how effectively they communicate with external devices. In this experiment, the focus is on implementing SPI (Serial Peripheral Interface) communication to control an LED Bar Graph module using the EFM8BB52 microcontroller.

2.1 The Serial Peripheral Interface (SPI)

SPI is a synchronous communication protocol that allows a master device (EFM8) to control slave peripherals (Bar Graph module). The communication uses four main lines:

- **MOSI (Master Out, Slave In)** – Sends data from master to slave.
- **MISO (Master In, Slave Out)** – Receives data from slave to master.
- **SCK (Serial Clock)** – Synchronizes data transfer.
- **CS (Chip Select)** – Enables the slave device when pulled low.

The EFM8 acts as the SPI master, generating clock pulses and sending data packets to control the LEDs.

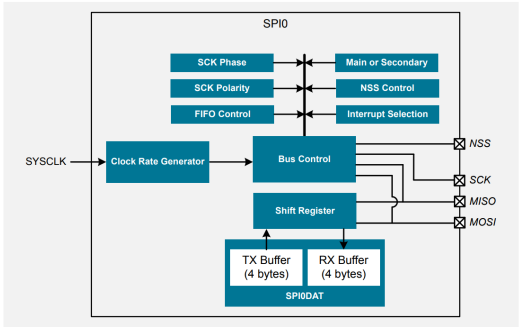

Figure 1: EFM8BB52 SPI0 Block Diagram (Source: EFM8BB52 Reference Manual, p. 277)

2.1.1 SPI Timing, Mode 0, and Bus Signal Operation

The EFM8BB52 microcontroller communicates with the Bar Graph Click module using SPI in **Mode 0**. This mode defines how the clock behaves and when data is sampled:

- **CPOL = 0:** Serial clock (SCK) idles LOW.
- **CPHA = 0:** Data on MOSI is sampled on the rising edge of SCK.

In the firmware, this configuration is implemented through:

```
SPI0CFG_CKPOL__IDLE_LOW
SPI0CFG_CKPHA__DATA_CENTERED_FIRST
```

SPI Bus Signals

The SPI interface uses four essential signals to control the Bar Graph Click board:

- **MOSI (Master Out, Slave In)** – carries data bits from the EFM8 to the bar graph module.
- **SCK (Serial Clock)** – synchronizes each data bit.
- **CS (Chip Select)** – enables the slave device; also latches the LED output when it transitions HIGH.
- **GND** – shared reference ground between both devices.

SPI Timing Diagram (Text Representation)

The following text diagram illustrates the behavior of SPI Mode 0 during a 16-bit transfer:

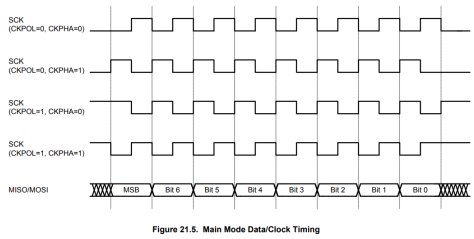

Figure 2: Main Mode Data/Clock Timing,
Source: EFM8BB52 Reference Manual, Figure 21.5

Explanation of Timing

- When **CS goes LOW**, the shift register inside the Bar Graph Click begins accepting data.
- On each rising edge of **SCK**, one bit is read from **MOSI**.
- After 16 clock cycles, the entire LED frame has been shifted in.
- When **CS transitions HIGH**, the shift register immediately latches the data and updates all LEDs simultaneously.

Why CS is Critical

The LED animation works correctly because of the following behavior:

- The shift register loads data only while **CS = LOW**.
- LED outputs update only when **CS rises to HIGH**.

If CS were toggled prematurely, the LED pattern would become corrupted. The firmware ensures correct timing by controlling the CS pin manually in each SPI transmission, providing clean and stable LED updates.

2.2 The Shift Register

The Bar Graph Click module uses a shift register to control 10 LEDs with few SPI pins. A shift register converts serial data (bits arriving one at a time) into parallel data (all 10 LEDs updating simultaneously).

It requires a specific 16-bit data packet. The rising edge of CS acts as a latch, telling the shift register to apply the received pattern to the LEDs.

2.3 Bar Graph Click – Hardware and Shift Register Operation

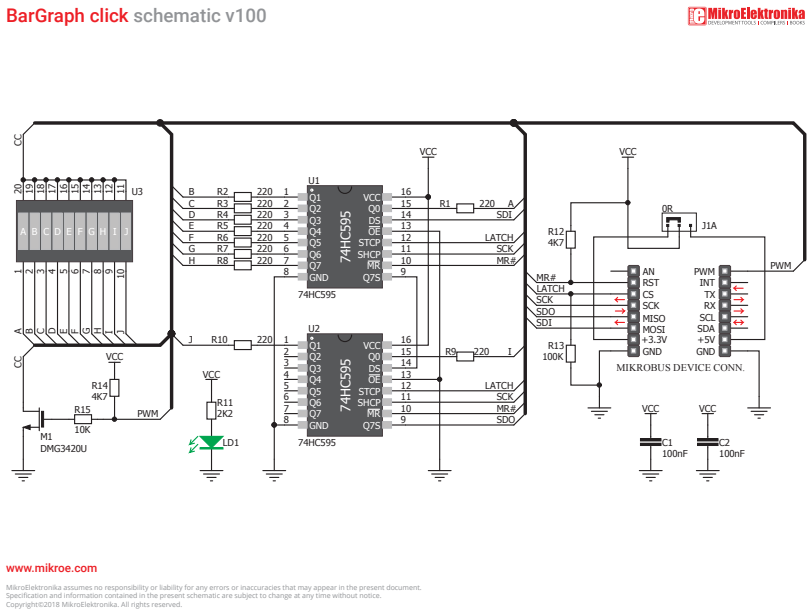

Figure 3: Schematic of the Bar Graph Click module showing two daisy-chained 74HC595 shift registers and the MikroBUS interface.
Source: https://www.mikroe.com/

The Bar Graph Click board contains a serial-in/parallel-out shift register, which is responsible for converting the SPI data stream from the microcontroller into 10 individual LED outputs.The schematic confirms the module uses two daisy-chained 74HC595 shift registers. The schematic also reveals a PWM-controlled MOSFET (M1) on the common cathode line, which allows for global brightness control of the LED array, though this feature is fixed to 'always on' in this specific firmware implementation.

Shift Register Architecture

The shift register includes the following key pins:

- **SER (Serial Data In)** – connected to MOSI.
- **SRCLK (Shift Clock)** – connected to SCK.
- **RCLK / LATCH (Strobe Input)** – connected to CS.
- **Q0–Q9 (Parallel Outputs)** – drive the 10 LEDs.

How It Works

- When **CS = LOW**, the shift register enters input mode.
- On every rising edge of **SCK**, one data bit from MOSI is shifted into the internal register.
- After 16 clock pulses (two bytes), the shift register holds a full 16-bit frame, where 10 bits correspond to the LED outputs.
- When **CS transitions HIGH**, the internal latch updates the LED drivers with the newly shifted data.

All LEDs update simultaneously on the rising edge of CS, which ensures a stable and flicker-free display.

LED Bit Mapping The firmware uses a 10-bit mask to represent the LED states:

$$\text{bit } 0 \rightarrow \text{LED } 1$$
$$\text{bit } 1 \rightarrow \text{LED } 2$$
$$\ldots$$
$$\text{bit } 9 \rightarrow \text{LED } 10$$

These 10 bits are split across two bytes to match the shift register's memory structure:

- **Byte 0:** upper two bits (LED 9–10)
- **Byte 1:** lower eight bits (LED 1–8)

This mapping matches the internal data arrangement of the Bar Graph Click board.

Summary

The Bar Graph Click does not update LEDs during shifting. The LED outputs are updated only when CS rises from LOW to HIGH, which prevents flickering and ensures that all LEDs change state simultaneously. This latching behavior is fundamental to producing clean animations on the bar graph display.

2.4 Project Objectives

- Configure the EFM8BB52's SPI0 peripheral as master (Mode 0).
- Map SPI and GPIO pins through the Crossbar configuration.
- Develop modular firmware using separate source files for SPI, bar graph control, and main logic.
- Display an animated LED pattern on the Bar Graph module.

3 Materials and Methods

3.1 Hardware Components

- EFM8BB52 Microcontroller (on development board)
- MikroE Bar Graph Click module
- PC for programming
- USB connection for power and debugging

3.2 Software Components

- IDE: Simplicity Studio 5
- Compiler: Keil C51 (integrated into Simplicity Studio)
- Protocol: SPI (Serial Peripheral Interface)

3.3 Methodology

The project was implemented in the following logical steps:

1. **Hardware Configuration (Port_Init):** The EFM8's pins were configured. The Crossbar was set to skip Port 0 (P0SKIP = 0xFF), mapping SPI0 to Port 1. SPI signals (SCK, MOSI) were set as outputs; CS and the Status LED were configured as push-pull outputs.
2. **Peripheral Initialization (SPI_Init, Timer3_Init):** SPI0 was configured as a 765 kHz, Mode 0 Master with manual 3-wire slave select. Timer3 was set up to generate a 1-second interrupt for a heartbeat status LED.
3. **Software Implementation (Layered):**

- **spi.c** – Provides SPI_WriteByte(), sending a byte and polling SPIF flag.
- **bar_control.c** – Handles Bar Graph protocol via Set_BarGraph_LEDs(), including CS pin management.
- **main.c** – Contains high-level logic for the bouncing light animation and the main loop.

3.4 Hardware Configuration & Pin Mapping

The EFM8 Crossbar was configured to enable SPI0 (XBR0 = XBR0_SPI0E__ENABLED;) and skip Port 0. Skipping P1.3 and P1.4 (P1SKIP) resulted in the following pin assignments:

EFM8 Pin	Assigned Function	Direction	Description
P1.0	SPI0_SCK	Output	SPI Clock, automatically controlled by SPI peripheral
P1.1	SPI0_MISO	Input	Master-In, Slave-Out (not used)
P1.2	SPI0_MOSI	Output	Master-Out, Slave-In (data from EFM8 to Bar Graph)
P1.3	GPIO (CS)	Output	Chip Select, manually controlled
P1.4	GPIO (LED)	Output	Status LED, manually controlled by Timer3 interrupt

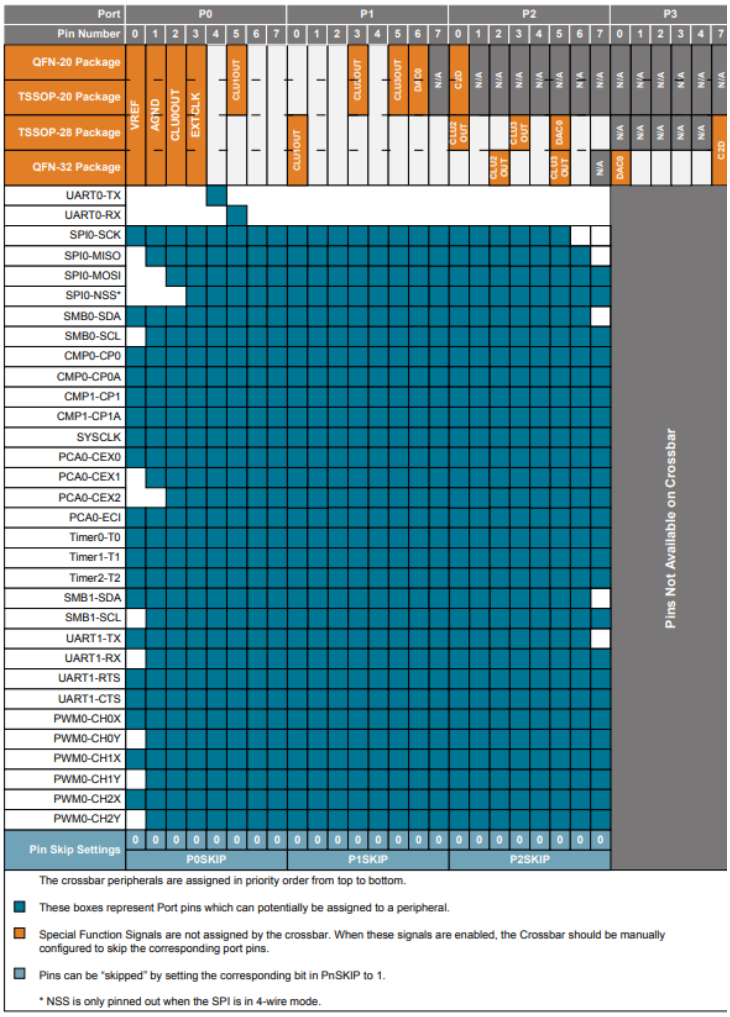

Figure 4: EFM8BB52 Crossbar Map showing SPI0 mapped to P1.0-P1.2 (Source: EFM8BB52 Datasheet, p. 65)

4 Result

The program was successfully compiled and flashed onto the EFM8BB52. Two different LED animation patterns were tested on the bar graph module through SPI communication.

4.1 Pattern 1: Heartbeat Pattern

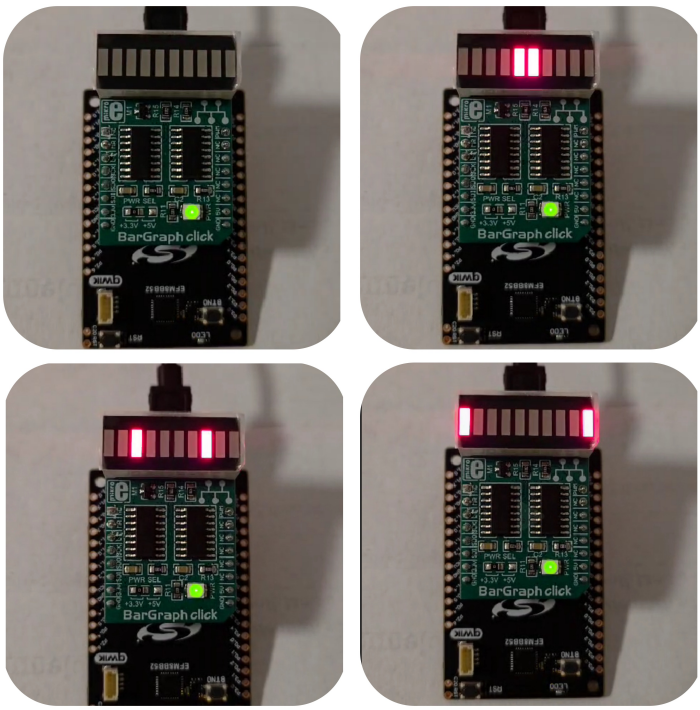

Figure 5: Heartbeat pattern

Figure 3 shows the heartbeat pattern, starting from the middle LEDs and expanding outward symmetrically. This verifies accurate timing and control of multiple LEDs through SPI.

4.2 Pattern 2: Bouncing Light

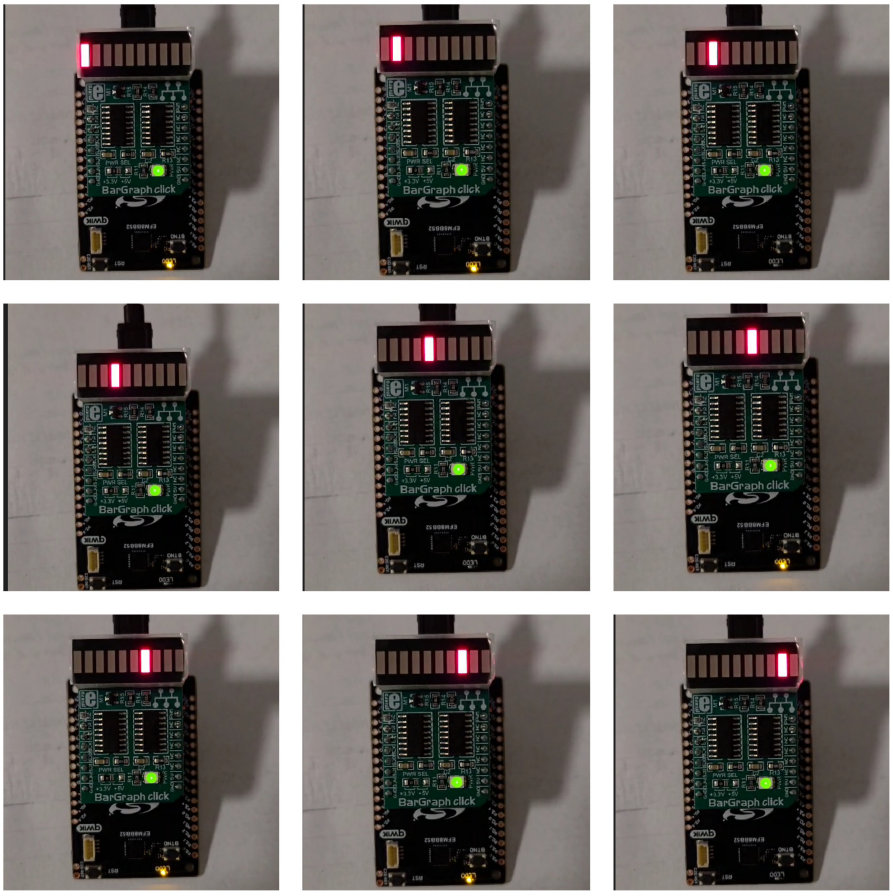

Figure 6: Bouncing Light

Figure 4 shows the bouncing light pattern, where a single LED moves back and forth across the 10-LED bar graph. This confirms that SPI communication and bit-shifting logic work correctly.

Summary: Both results confirm that the SPI interface and firmware design function reliably, successfully controlling the LED bar graph with different animation patterns.

4.3 Firmware Operation of `barcontrol.c`

The file `barcontrol.c` is responsible for converting a 10-bit LED mask into the correct 16-bit SPI data frame required by the Bar Graph Click module. It encapsulates the low-level communication protocol by preparing two data bytes, transmitting them via SPI, and generating the required chip-select (CS) strobe to latch the LED outputs.

Function Overview

```
void Set_BarGraph_LEDs(uint16_t mask)
{
    uint8_t bargraph_data[2];

    bargraph_data[0] = (uint8_t)((mask >> 8) & 0x03); // LEDs 9{10
    bargraph_data[1] = (uint8_t)(mask & 0xFF);        // LEDs 1{8

    CS = 0;
    SPI_Transmit(bargraph_data, 2);
    CS = 1;
}
```

Step-by-Step Explanation

1. Mask Splitting

The LED pattern is stored as a 10-bit value. For example:

$$\texttt{mask} = 0b0000001101$$

This 10-bit value must be split into two bytes to match the shift register layout:

- **Byte 0** contains the upper two bits (LED 9–10)

- **Byte 1** contains the lower eight bits (LED 1–8)

2. Start SPI Transfer

Setting **CS = 0** places the shift register into input mode, allowing data to be shifted in.

3. Transmit Two Bytes Over SPI

The function $\texttt{SPI}_Transmit()sendsthetwo-byteframeMSB-first.EachbyteisclockedintotheBarG$

```
4.  Latch the Output
When CS = 1, the shift register output latches immediately.  This
causes all 10 LEDs to update simultaneously, preventing flicker and
ensuring a stable display.
```

How the Program Generates the Animation

```
In the main application, the 10-bit mask is shifted left and right:

0000000001
0000000010
0000000100
...
0100000000
1000000000
(reverse)

   At each step:

  1. A new LED position is encoded in the mask.

  2. Set_BarGraph_LEDs() sends the updated frame.

  3. The latch pulse (CS = 1) refreshes the LED outputs.

   This produces the ‘‘bouncing light’’ effect across the 10 LEDs.
```

Summary

The barcontrol.c file handles:

- LED mask formatting,
- SPI data framing,
- shift register communication,
- and LED output latching.

It serves as an abstraction layer between the animation logic in main.c and the SPI hardware driver, enabling clean and modular control of the Bar Graph Click module.

5 Discussion

The experiment successfully demonstrated SPI communication between the EFM8BB52 microcontroller and the LED Bar Graph Click module. The results showed two different LED animation patterns: the bouncing light and the heartbeat pulse. These confirm that the SPI interface correctly transfers data to the shift register, and the LED outputs respond as expected.

The bouncing pattern verified that the bit-shifting logic works correctly, while the heartbeat pattern demonstrated precise timing control and multi-LED activation. Together, these prove the reliability and flexibility of the SPI communication and the firmware design.

Beyond these two animations, many other LED sequences could be created, such as wave effects, alternating flashes, or sensor-based indicators. This showcases how modular SPI control can be extended for creative or practical purposes.

The project also demonstrates potential applications for LED bar displays, such as visual indicators for system status, sensor outputs, or user feedback in embedded devices. Overall, the experiment confirms that a structured SPI communication setup can effectively control external peripherals like LED arrays in real-time applications.

Appendix: Full Source Code

6.1 main.c (Application Layer)

```
#include "main.h"

void SiLabs_Startup (void)
{
    IE_EA = 0;
    WDTCN = 0xDE;
    WDTCN = 0xAD;
    Port_Init();
    Timer3_Init();
    SPI_Init();
    XBR2 = XBR2_XBARE__ENABLED;
    IE_EA = 1;
}

void Port_Init(void)
{
    P0SKIP = 0xFF;
    P1SKIP = (1 << 3) | (1 << 4);

    P1MDIN  |= (1 << 4) | (1 << 3);
    P1MDOUT |= (1 << 4) | (1 << 3);
    CS = 1;

    P1MDIN  |= (1 << 0) | (1 << 1) | (1 << 2);
    P1MDOUT |= (1 << 0) | (1 << 2);
    P1MDOUT &= ~(1 << 1);

    XBR0 = XBR0_SPI0E__ENABLED;
    XBR1 = XBR1_PCA0ME__DISABLED;
```

```
}

void Delay_ms(uint16_t ms)
{
    uint16_t i, j;
    for(i = 0; i < ms; i++)
        for(j = 0; j < 300; j++) _nop_();
}

void Timer3_Init(void)
{
    if ((LFO0CN & LFO0CN_OSCLEN__BMASK) == 0)
    {
        LFO0CN |= LFO0CN_OSCLEN__ENABLED;
        while ((LFO0CN & LFO0CN_LFORDY__BMASK) == 0);
    }

    TMR3CN0 &= ~TMR3CN0_T3SPLIT__BMASK;
    TMR3CN0 |= TMR3CN0_T3XCLK__LFOSC_DIV_8;

    TMR3RLL = 0xF0;
    TMR3RLH = 0xD8;
    TMR3L   = 0xF0;
    TMR3H   = 0xD8;

    EIE1 |= EIE1_ET3__ENABLED;
    TMR3CN0 |= TMR3CN0_TR3__BMASK;
}

SI_INTERRUPT(TIMER3_ISR, TIMER3_IRQn)
{
    TMR3CN0 &= ~TMR3CN0_TF3H__BMASK;
```

```
    LED = !LED;
}

int main (void)
{
    uint16_t led_mask = 0x0001; // Start with LED 1
    bool going_right = true;



    while (1)
    {

        Set_BarGraph_LEDs(led_mask);

        // Delay for 50ms
        Delay_ms(50);

        if (going_right)
        {
            led_mask <<= 1; // Shift to the next LED
            if (led_mask > (1 << 9)) // After LED 10 (
               bit 9)
            {
                going_right = false;
                led_mask = (1 << 8); // Go to LED 9
            }
        }
        else
        {
            led_mask >>= 1; // Shift to the previous LED
            if (led_mask == 0) // Before LED 1
```

```
            {
                going_right = true;
                led_mask = 0x0002; // Go to LED 2
            }
        }
    }
}
```

6.2 spi.c (Driver Layer)

```
/*
 * spi.c
 */

#include <SI_EFM8BB52_Register_Enums.h>

void SPI_Init(void)
{
    SPI0CFG = SPI0CFG_MSTEN__MASTER_ENABLED |
              SPI0CFG_CKPHA__DATA_CENTERED_FIRST |
              SPI0CFG_CKPOL__IDLE_LOW;

    SPI0CN0 &= ~(SPI0CN0_NSSMD__FMASK);
    SPI0CN0 |= SPI0CN0_NSSMD__3_WIRE;
    SPI0CKR = 1;
    SPI0CN0 |= SPI0CN0_SPIEN__ENABLED;
}

uint8_t SPI_WriteByte(uint8_t tx_byte)
{
    SPI0CN0_SPIF = 0;
    SPI0DAT = tx_byte;
```

```
    while(SPI0CN0_SPIF == 0);
    return SPI0DAT;
}

void SPI_Transmit(uint8_t* buffer, uint8_t length)
{
    uint8_t i;
    for (i = 0; i < length; i++)
    {
        SPI_WriteByte(buffer[i]);
    }
}
```

6.3 bar_control.c (Abstraction Layer)

```
/*
 * bar_control.c
 */
#include "spi.h"
#include <stdint.h>
#include "main.h"

void Set_BarGraph_LEDs(uint16_t mask)
{
    uint8_t bargraph_data[2];

    bargraph_data[0] = (uint8_t)((mask >> 8) & 0x03); //
        LEDs 9-10
    bargraph_data[1] = (uint8_t)(mask & 0x00FF);      //
        LEDs 1-8

    CS = 0;
```

```
    SPI_Transmit(bargraph_data, 2);
    CS = 1;
}
```